

Billing Application - Database Architecture & Data Flow Report

Comprehensive Guide to Data Sources, Storage Tables, and Flow Mechanics

1. Executive Summary

This document provides an exhaustive specification of the database architecture, data origin points, and relational database storage tables for the Restaurant & Retail Billing Application. The system operates with a multi-tenant backend architecture (Django REST Framework) complemented by an offline SQLite cache engine in the Flutter mobile/desktop client.

2. Module-wise Data Origin & Database Storage Mapping

Module / Data Type	Data Source / Origin	Database Table(s)	Key Stored Fields & Description
Customer Details (Customer Profile, Address)	- Billing Screen Customer Autocomplete - Customer Management UI	Backend: customers_customer	id, name, mobile_number (Unique Key), address, gst_number, status, created_at, updated_at
Customer Ledger & Payments (Payment History, Due Amounts)	- Customer Ledger Pay Due UI - Credit Bill Partial Downpayments	Backend: customers_customerpayment	id, customer_id (FK), token_id (FK), payment_number (P0001...), amount, payment_mode (cash/upi), date, note, created_at
Bills & Tokens (Current Bill, Receipts)	- POS Billing Screen - Token Generation Form	Backend: tokens_token Local Cache: tokens (local_billing.db)	token_number, bill_number (0001...), date, order_type (dine_in/takeaway/delivery), status, subtotal, gst_amount, discount, total, received_amount, balance_due, is_paid, payment_mode
Menu & Items (Food Items, Categories)	- Menu Management UI - Category Management Screen	Backend: menu_category menu_menuitem Local Cache: items (local_billing.db)	Category: name, icon, sort_order, is_active Menuitem: category_id (FK), name, description, price, item_type (veg/non_veg/egg), is_available, image
User Auth & Shop Profile (User Credentials, Shop Info)	- Registration Screen - OTP & Password Login	Backend: core_user core_shopsettings	phone (Unique), name, email, shop_name, account_status (approved/trial/pending), permissions, trial_start, trial_end
Analytics & Reports (Sales Summary, Customer Data)	Generated dynamically from existing transactions	Dynamic Query: No separate table; computed directly from tokens & payments	Real-time aggregated metrics: Total Sales, Customer Spend Summary, Daily Item Sales, Customer Net Due Statement

3. Data Synchronization & Storage Architecture

- 1. Multi-Tenant Backend Storage:** All persistent business data is saved into Django's multi-tenant backend relational database. Each shop operates on an isolated schema or tenant configuration ensuring complete data isolation and privacy.
- 2. Offline-First Local Cache:** The Flutter frontend utilizes SQLite (local_billing.db) via sqflite to cache menu items, local bill history, and shop settings. This ensures instant billing generation even during transient network drops.

3. Real-time Customer Ledger Synchronization: Customer Net Due balances are computed dynamically from both total bill debits and direct customer payments (CustomerPayment records). Synchronization event listeners in Flutter guarantee instant UI updates across all screens.